GUIDE 26 · AGENTS

Read structured MCP results properly

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=26-mcp-structured-results. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** an MCP client that reads QueryAPIGate's results as data - rows, truncation and error codes - instead of parsing text meant for an LLM.

Every QueryAPIGate tool returns its result twice: as `structuredContent` (a JSON object matching the tool's declared `outputSchema`) and as a text block holding the same object serialized. Read `structuredContent`. The text is there for clients that can't.

The shapes

Declared by every tool, verified with `tools/list`:

TOOL	`OUTPUTSCHEMA`
a saved query, `execute_sql`	`{"rows": [object, ...], "truncated": boolean}`
`list_tables`	`{"tables": [object, ...], "truncated": boolean}`

The envelope is the same for every saved query. The schema doesn't describe each column - a query's columns are only known once it runs - so treat each row as an object keyed by column name.

A saved query's **input** schema, by contrast, is specific: it's built from the query's parameter rules. For `example_top_films`:

```json
{"type": "object", "required": [], "properties": {
  "category": {"type": "string", "enum": ["Action", "Comedy", "Documentary", "Drama", "Family", "Sci-Fi"]},
  "top_n": {"type": "integer", "minimum": 1, "maximum": 50, "default": 10}}}
```

and its description is the saved query's own: *Most-rented films, optionally within one category.* Good parameter rules and descriptions (see Use bound parameters safely) are what an LLM reads to call the tool correctly.

Success

```python
res = await session.call_tool("example_top_films", {"top_n": 3})
res.isError            # False
res.structuredContent  # {"rows": [{"title": "Electric Signal", "category": "Comedy", "rating": "PG",
                       #            "rentals": 977, "rank": 1}, ...], "truncated": False}
```

`truncated`: there's more than you got

A tool call returns at most `QUERYAPIGATE_MCP_MAX_ROWS` rows (default 200) - an LLM's context can't hold a whole table. Verified with the cap set to 5:

CALL	ROWS	TRUNCATED
`example_top_films` with `top_n: 3`	3	`false`
`example_top_films` with `top_n: 8`	5	`true` - the cap cut it
`example_top_films` with `top_n: 8, page_size: 2`	2	`true` - the caller asked for fewer

`page_size` lowers the cap for one call; it can't raise it above `QUERYAPIGATE_MCP_MAX_ROWS`. When `truncated` is `true`, tell the model so (or have it narrow the question - a `WHERE`, a `LIMIT`, an aggregate) rather than letting it treat the rows as the whole answer. For a full export, use REST's `?stream=true` or `queryapigate export` instead.

Errors

A refused or failed call has `isError: true`, and `structuredContent` carries the same `error` and `code` a REST error body has:

```python
res.isError            # True
res.structuredContent  # {"error": "This API key may only query these tables: main.customers,
main.orders,
                       #  main.products. Forbidden: main.salaries.", "code": "table_not_allowed"}
```

Branch on `code`; the `error` text is for people and may change between releases. Codes seen over MCP while writing this guide:

CODE	MEANING	WHAT THE CLIENT CAN DO
`rate_limited`	over the key's or server's limit; `retry_after` gives seconds	wait `retry_after`, then retry
`table_not_allowed`	`allowed_tables` forbids a table	ask without that table
`read_only`	a write through `execute_sql`	don't retry - writes aren't possible over MCP
`multiple_statements`	more than one statement	send one at a time
`connection_forbidden`	the key can't use that connection	use one it can
`secret_key_required`, `password_undecryptable`	the server can't open the connection	tell a human - nothing the agent can fix

The full list is under Errors. A rate-limited call, verified with a `2/minute` key's third call:

```json
{"error": "Rate limit exceeded for this API key - retry after 30s", "code": "rate_limited",
"retry_after": 30}
```

One exception: invalid arguments. If arguments don't match the tool's input schema - `"status": "refunded"` where the schema says `enum: ["paid", "pending"]` - the MCP layer rejects the call before QueryAPIGate runs it: `isError: true`, the text *Input validation error: 'refunded' is not one of ['paid', 'pending']*, and `structuredContent` is `null`. Handle a `null` `structuredContent` on an error as "the arguments were wrong".

A small reader

```python
import json


class ToolFailed(Exception):
    def __init__(self, code, message, retry_after=None):
        super().__init__(f"{code}: {message}")
        self.code, self.retry_after = code, retry_after


def read_result(res):
    """(rows, truncated) for a success; raises with the code for a failure."""
    data = res.structuredContent
    if res.isError:
        code = (data or {}).get("code", "invalid_arguments")
        message = (data or {}).get("error") or (res.content[0].text if res.content else "")
        raise ToolFailed(code, message, (data or {}).get("retry_after"))
    if data is None:                                  # a client or server without structured output
        data = json.loads(res.content[0].text)
    return data.get("rows", data.get("tables")), data["truncated"]
```

Next steps

- Let an agent explore your schema and run ad-hoc SQL.
- Let an AI agent call your saved queries via MCP.
- MCP server reference.